

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Фронт-энд разработка

Отчет

Лабораторная работа

Выполнила:

Шадрина Александра

Группа К3342, поток ФРЭНД 1.1

Проверил:

Добряков Д. И.

Санкт-Петербург

2024 г.

Задача

Мигрировать ранее написанный сайт на фреймворк Vue.JS.

Вариант – Сервис для обмена рецептами и кулинарных блогов.

- Вход
- Регистрация
- Личный кабинет пользователя (сохраненные рецепты, публикации)
- Поиск рецептов с фильтрацией по типу блюда, сложности, ингредиентам
- Страница рецепта с фото, пошаговыми инструкциями и видео
- Социальные функции (комментарии, лайки, подписки на кулинаров)

Минимальные требования:

- Должен быть подключён роутер
- Должна быть реализована работа с внешним API
- Разумное деление на компоненты
- Использование composable

Ход работы

Работа с внешним API

После выполнения ДЗ 5 был начат проект на Vue.js. Дальнейшая работа проходила по гайду. Необходимо было совместить на фреймворке имеющуюся верстку и внешний API, написанный ранее с помощью DRF. Так, внутри компонентов Vue.js данные, полученные с сервера, должны отрисовываться в соответствии с имеющейся версткой. Для начала для серверной части был настроен CORS, чтобы клиент мог получать данные с нее. Далее в файле `api/instance.js` проекта была прописана ссылка на серверную часть (`const apiURL = 'http://127.0.0.1:8000'`), которая затем

использовалась в каждом компоненте с дописанным соответствующим эндпоинтом в формате:

```
import instance from "@/api/instance";

try {

  await instance.post("/auth/users/", {

    username: formData.value.username,

    email: formData.value.email,

    password: formData.value.password,

  });

}
```

Роутинг

Для каждого написанного компонента в файле роутера была прописана ссылка в формате:

```
import { createRouter, createWebHistory } from 'vue-router'

import Profile from "@/views/Profile.vue";

const router = createRouter({

  history: createWebHistory(import.meta.env.BASE_URL),

  routes: [

    {

      path: '/profile',

      name: 'Profile',

      component: Profile,

      meta: { requiresAuth: true },

    },

  ],

})

router.beforeEach((to, from, next) => {

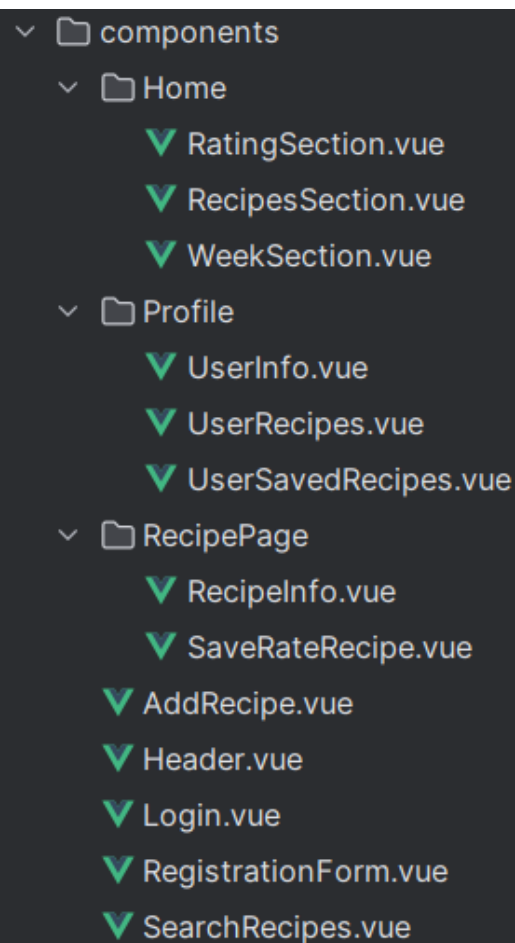
  const token = localStorage.getItem("auth_token");
```

```
if (to.matched.some(record => record.meta.requiresAuth) && !token) {  
  next({ name: 'Login' });  
} else {  
  next();  
}  
});  
  
export default router
```

В данном листинге продемонстрирован защищенный роутинг, примененный к компоненту профиля пользователя: на эту страницу может перейти только авторизованный пользователь.

Деление на компоненты

По окончании работы деление на компоненты оказалось следующим:



```
components  
├── Home  
│   ├── RatingSection.vue  
│   ├── RecipesSection.vue  
│   └── WeekSection.vue  
├── Profile  
│   ├── UserInfo.vue  
│   ├── UserRecipes.vue  
│   └── UserSavedRecipes.vue  
├── RecipePage  
│   ├── RecipeInfo.vue  
│   ├── SaveRateRecipe.vue  
│   ├── AddRecipe.vue  
│   ├── Header.vue  
│   ├── Login.vue  
│   ├── RegistrationForm.vue  
│   └── SearchRecipes.vue
```

Так, было создано 3 представления для основных страниц приложения – главная, страница рецепта и профиль пользователя. В каждом из них были перечислены используемые компоненты, разложенные по соответствующим папкам. Также были созданы компоненты регистрации и авторизации, шапка сайта и добавление рецепта на сайт.

Composable

В приложение было внедрено использование composable. Опишем пример для главной страницы, состоящей из 3 секций – рецепты, добавленные за последнюю неделю; рецепты, рейтинг которых выше 8.0; все рецепты, добавленные на сайт. Загрузка рецептов с сервера была помещена в composable и затем использована в каждом из компонентов, соответствующих перечисленным секциям в формате:

```
import { onMounted } from 'vue';

import { useRecipes } from '@/composables/useRecipes';

const { recipes, loading, error, fetchRecipes } =
useRecipes('/recipes/');

onMounted(async () => {

  await fetchRecipes();

});
```

После перечисления удовлетворенных минимальных требований посмотрим на основные страницы сайта. Все данные подгружаются с сервера. Также каждый пользователь имеет возможность добавлять изображения профиля и рецептов, которые тоже подгружаются на сайт; в противном случае высвечивается заглушка.



На этой неделе



Лосось, тушенный в сливках



Пирог с грушей



Лимонный пирог

Рейтинговые рецепты



Согревающий тыквенный суп

Автор: alexa

500 г тыквы 1 морковь 2 картофелины 200 мл сливок Соль и перец по вкусу Семечки для подачи

localhost:5173/recipes/10





Вишневый пирог

Автор: tatyasaul

Вишня без косточек, масло сливочное

Все рецепты



Утка по-пекински

Автор: alexa

Ингредиенты:
вкусная утка



Блины

Автор: george

Ингредиенты:
блины со
сгущенкой



картофель по-деревенски

Автор: alexa

Ингредиенты:
картофель



утка

Автор: valera

Ингредиенты: утка





Рисунок 1 – Главная страница

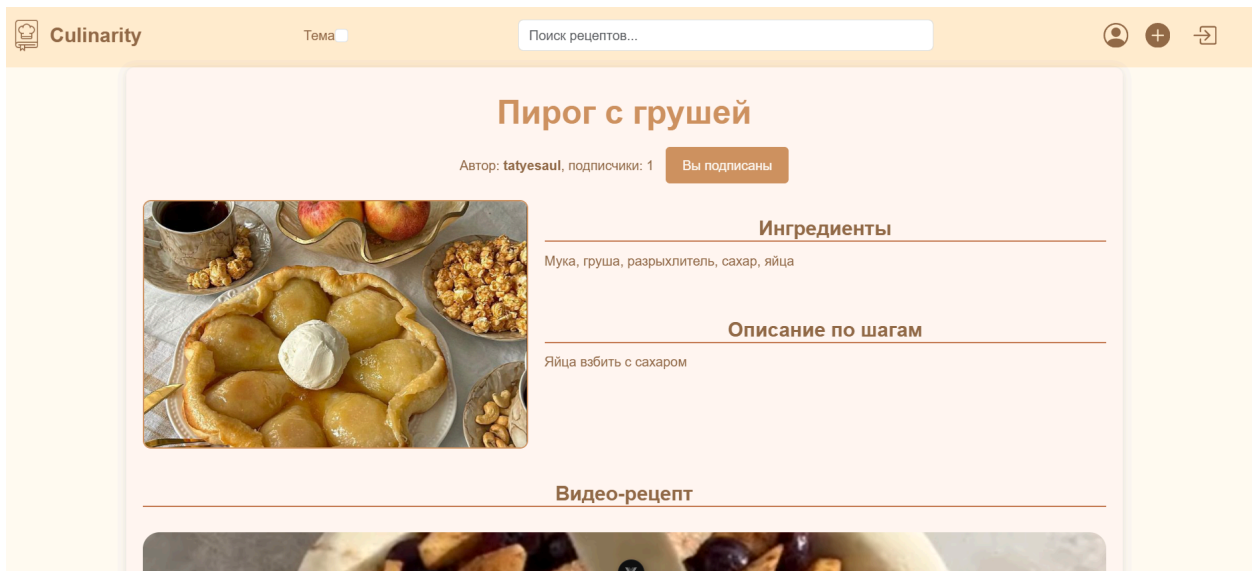


Рисунок 2 – Страница рецепта



Профиль пользователя



Личная информация

Имя: alexa

Email: alexasha2604@gmail.com

Дата регистрации: 02.12.2024

Подписчики: 1

Віо: обожаю готовить!!!

Редактировать профиль

Удалить профиль

Загрузить изображение

Выберите файл

Файл не выбран

Мои рецепты



Утка по-пекински
вкусная утка



картофель по-деревенски
картофель



Согревающий тыквенный суп
500 г тыквы
1 морковь
2 картофелины
200 мл сливок
Соль и перец по вкусу
Семечки для подачи

Сохранённые рецепты



Лосось, тушенный в сливках
Автор: tatyasaul
Лосось
сливки 10%
зелень
специи



Пирог с грушей
Автор: tatyasaul
Мука
груша
разрыхлитель
сахар
яйца



Вишневый пирог
Автор: tatyasaul
Вишня без косточек
масло сливочное

Рисунок 3 – Профиль пользователя

Добавить рецепт

Название

Ингредиенты

Инструкции

Сложность

Теги

иностранная еда
традиционная русская кухня
мексиканская еда



Рисунок 4 – Добавление рецепта

Вывод

Верстка и серверная часть, написанная с помощью Django Rest Framework и Djoser, были совмещены с помощью фреймворка Vue.js. Все минимальные требования были удовлетворены.